

Editor de texto en moon

Trabajo de Sistemas Operativos de Pablo Manjarres y Valentina Barbosa. El código está en `cat_editor.c`; se entra desde el shell con `edit`.

El editor integra en el shell de clase los cinco comandos base y la inserción y búsqueda exigidas para dos integrantes. Usa llamadas POSIX para manejar archivos. La vista con cursor y la barra permiten editar y ejecutar órdenes sin ocultar el texto.

Uso

`edit archivo.txt` abre el archivo en pantalla completa con líneas numeradas. El pie muestra los siete comandos. `Ctrl+L` enfoca una barra sin ocultar el archivo; se escribe una orden y se pulsa Enter. El resultado aparece junto al documento y la barra queda lista para la siguiente orden. `Ctrl+L` vuelve a la edición con el cursor. Escape cancela la orden pendiente y devuelve el foco al documento.

Las órdenes `a`, `d` e `i` escriben sus cambios en el archivo y actualizan la vista. El texto escrito directamente con el cursor se guarda con `Ctrl+O`. Si hay cambios pendientes, hay que guardarlos antes de ejecutar las órdenes de archivo; abrir la barra no los guarda ni los descarta.

`edit` sin nombre de archivo conserva el modo de líneas. Su ayuda aparece al entrar y se puede repetir con `help` o `?`, incluso sin archivo abierto. `v` abre la vista visual. Allí, `Ctrl+X` regresa al modo de líneas; `q` cierra el archivo y vuelve a moon; si hay cambios del cursor pendientes, primero exige guardarlos con `Ctrl+O`. Si se entró directamente con `edit archivo.txt`, `Ctrl+X` vuelve a moon. `Ctrl+X` descarta los cambios del cursor que no se hayan guardado.

Modo de líneas	Qué hace	Llamadas usadas
<code>o</code> archivo	Abre o crea el archivo	<code>open</code> , <code>close</code> al cambiar de archivo
<code>p</code> [n]	Imprime todo o la línea n	<code>lseek</code> , <code>read</code> , <code>write</code>
<code>a</code> texto	Añade una línea	<code>lseek</code> , <code>read</code> , <code>write</code>
<code>d</code> n	Borra la línea n	<code>lseek</code> , <code>read</code> , <code>write</code> , <code>ftruncate</code>
<code>q</code>	Cierra el archivo y vuelve a moon	<code>close</code>
<code>i</code> n texto	Inserta una línea en n	<code>lseek</code> , <code>read</code> , <code>write</code> , <code>ftruncate</code>
<code>s</code> palabra	Muestra las líneas que contienen la palabra	<code>lseek</code> , <code>read</code>

Integración con el shell

`main.c` ya despacha comandos desde `commands[]` en `registry.c`. Añadimos `edit` a esa tabla y declaramos `cmd_edit` en `shell.h`. No hubo que cambiar el ciclo principal ni el parser.

La categoría `data` agrupa operaciones que terminan tras una llamada. El editor toma la entrada hasta que el usuario sale y mantiene abierto un descriptor entre órdenes. Por eso tiene su propia categoría:

```
/* antes: última categoría */
{ "util", "Utilities", "getuid . time" },

/* ahora: una fila más en categories[] */
{ "editor", "Text editor", "open . read . write . lseek . ftruncate . termios" },
```

`q` retorna de `cmd_edit` después de cerrar el descriptor. No llama a `exit()`: hacerlo también cerraría el shell que contiene al editor.

Archivos y memoria

El modo de líneas guarda el descriptor en `ed_fd`. Lee el archivo completo para localizar líneas y, después de `d` o `i`, escribe el resultado desde el inicio y ajusta su longitud con `ftruncate`. En pantalla completa, `Doc` guarda un arreglo dinámico de líneas; `Ctrl+O` las reúne y escribe el archivo. La barra y el modo de líneas comparten `ed_execute` y las mismas operaciones de archivo y la protección de cambios sin guardar, evitando duplicar las reglas de las órdenes. `tcgetattr` y `tcsetattr` permiten leer teclas una a una y restaurar el terminal al salir.

`Doc` usa `char **line` para acceder por índice. `count` registra las líneas usadas y `cap` los espacios reservados: empieza con 32 y duplica la capacidad cuando se llena, evitando ampliar el arreglo por cada línea. Cada línea tiene su propia reserva para sustituirla sin copiar el documento. En `ed_visual`, `Doc` vive en la pila (`stack`); el arreglo y las líneas están en el `heap` y se liberan al salir. Las operaciones de archivo usan un búfer continuo para desplazar y escribir bytes.

La barra reserva 2048 bytes para entrada y 4096 para resultados. Indica los resultados recortados; el desplazamiento vertical permite recorrer el documento con la barra visible. Estos límites acotan la memoria de la interfaz; el documento usa memoria dinámica.

Se eligió cargar el texto en memoria porque simplifica las operaciones por línea. Editar solo en disco usaría menos RAM, pero complicaría mover la cola del archivo. Un índice de offsets evitaría releer todo para `p`, pero habría que actualizarlo tras cada inserción o borrado. Aquí se pagan lecturas y escrituras completas, una decisión razonable para archivos pequeños.

```
d 2: [uno\n][dos\n][tres\n] -> [uno\n][tres\n]
      destino <---- origen memmove admite el solapamiento

i 2: [uno\n][tres\n] + [dos\n] -> [uno\n][dos\n][tres\n]
      se copia a otro búfer; origen y destino no se solapan
```

Errores y límites

Las llamadas de archivo y las reservas de memoria comprueban sus retornos; los fallos de sistema se muestran con `perror`. Si falla o con otro nombre, el archivo anterior sigue abierto. Se usan `open`, `read`, `write`, `lseek`, `ftruncate` y `close` para el archivo, sin `fopen`, `fread`, `fwrite` ni `fclose`.

El archivo debe caber en memoria. El guardado reescribe el archivo en su lugar, sin copia temporal, así que no es atómico. La vista completa acepta teclas imprimibles ASCII; no es un editor Unicode. Los cambios de esa vista se guardan solo con `Ctrl+O`.

Pruebas

En Linux: `make && sh tests/run_editor_tests.sh`. El script comprueba los comandos de ambos niveles, fallos de apertura, archivos sin salto final y edición visual mediante una terminal seudográfica. También verifica que la barra mantenga el archivo visible, que las órdenes actualicen su contenido, que los errores conserven el archivo anterior, la protección del texto sin guardar y la restauración del terminal al salir.